

PRESENTATION_DEMO_SCENARIO.md

발표 목표

이 문서는 **ixissage** 수업 발표에서 보여줄 데모 흐름을 정리한다.

발표의 핵심 메시지는 다음과 같다.

- **ixissage**는 룰 기반 스미싱 탐지 앱이 아니다.
- 문자 본문을 AI Transformer 모델이 **normal** 또는 **smishing**으로 분류한다.
- Android 앱은 모델 확률인 **normalProbability**, **phishingProbability**, **predictedLabel**을 표시한다.
- 정상, 주의, 스팸 경고 배지는 텍스트 키워드가 아니라 **phishingProbability**를 시각화한 UI 표시다.
- 현재 발표 버전은 Android 온디바이스 실시간 추론 대신 Python 모델 추론 결과를 **sample_messages.json**에 저장해 보여준다.

한 줄 요약

Python에서 학습된 한국어 문자 스미싱 Transformer 모델이 확률을 계산하고, Android Compose 앱은 그 확률을 classifier 인터페이스를 통해 받아 목록과 상세 화면에 표시한다.

발표 전 준비

프로젝트 루트:

```
cd /Users/topjs/jstop/codexixissage
```

Python 의존성:

```
python3 -m pip install -r requirements.txt
```

Android 빌드 환경 예:

```
export  
JAVA_HOME=/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home  
export ANDROID_HOME=/opt/homebrew/share/android-commandlinetools  
export ANDROID_SDK_ROOT=$ANDROID_HOME
```

현재 준비된 핵심 파일:

- **models/smishing_classifier/**
- **src/inference.py**
- **outputs/sample_messages.json**

- android/IxissageApp/app/src/main/assets/sample_messages.json
- android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/SmishingClassifier.kt
- android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/PrecomputedSampleClassifier.kt

데모 흐름

1. 프로젝트 목적 설명

예상 시간: 30초

말할 내용:

이 프로젝트는 실제 보안 제품이 아니라 인공지능 과목 수업 프로젝트입니다. 목표는 문자 메시지를 AI 모델로 정상/스미싱 분류하고, Android 앱에서 모델 확률을 사용자에게 보여주는 것입니다.

강조할 제약:

- 룰 기반 탐지는 하지 않는다.
- 특정 키워드나 URL 포함 여부로 위험 판단하지 않는다.
- 입력 feature는 문자 본문 text뿐이다.
- 판단 근거는 모델의 softmax probability다.

2. 모델 학습 결과 설명

예상 시간: 1분

보여줄 파일:

- outputs/baseline_report.md
- outputs/transformer_report.md
- MODEL_PLAN.md

말할 내용:

먼저 TF-IDF + Logistic Regression으로 baseline을 만들고, 그 다음 monologg/koelectra-small-v3-discriminator 기반 Transformer를 fine-tuning했습니다. 두 모델 모두 문자 본문만 입력으로 사용했고, 사람이 만든 위험 키워드 feature는 넣지 않았습니다.

현재 Transformer test 결과:

- accuracy: 0.998513
- precision: 0.999130
- recall: 0.998262
- F1: 0.998696
- confusion matrix: normal->normal 866, normal->smishing 1, smishing->normal 2, smishing->smishing 1149

짧게 설명할 포인트:

- baseline보다 Transformer F1이 높다.

- false positive/false negative 예시를 분석했지만, 그 예시에서 키워드 규칙을 만들지는 않았다.

3. Python 단일 문자 추론 시연

예상 시간: 1분

실행:

```
python3 src/inference.py --text "오늘 저녁에 회의 끝나고 집에 갈게." --include-ui-badge
```

출력에서 볼 항목:

- predictedLabel
- normalProbability
- phishingProbability
- uiBadge

말할 내용:

이 단계에서는 Android에 붙이기 전에 Python에서 학습된 모델이 하나의 문자에 대해 확률을 출력하는지 확인합니다. 결과는 keyword if문이 아니라 Transformer 모델의 softmax 결과입니다.

예시 5개를 한 번에 보여주려면:

```
python3 src/inference.py --examples --include-ui-badge
```

Markdown 예시 결과 저장:

```
python3 src/inference.py --examples --write-examples --include-ui-badge
```

생성 파일:

- outputs/inference_examples.md

4. Android 샘플 JSON 설명

예상 시간: 1분

보여줄 파일:

- outputs/sample_messages.json
- android/IxissageApp/app/src/main/assets/sample_messages.json

JSON 필드:

- id

- sender
- body
- groundTruthLabel
- predictedLabel
- normalProbability
- phishingProbability

말할 내용:

Android 발표 버전은 실제 SMS 권한을 쓰지 않습니다. test split에서 샘플을 뽑고, 개인정보처럼 보이는 값과 실제 위험 URL처럼 보이는 값은 마스킹합니다. 그 다음 Python Transformer 모델로 추론해서 확률을 JSON에 저장합니다.

샘플 JSON을 새로 만들 때:

```
python3 scripts/create_sample_messages.py
```

Android assets를 바로 갱신하고 싶을 때:

```
python3 scripts/create_sample_messages.py \
  --output android/IxissageApp/app/src/main/assets/sample_messages.json
```

주의:

- 마스킹은 개인정보 보호와 샘플 안전을 위한 처리다.
- 마스킹 로직은 위험 점수를 만들지 않는다.
- 확률은 마스킹된 문자 본문에 대해 모델이 계산한 값이다.

5. Android 앱 시연

예상 시간: 1분 30초

빌드:

```
cd android/IxissageApp
./gradlew build
```

기기나 에뮬레이터에 설치:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Android Studio에서 실행할 때:

- android/IxissageApp 폴더를 연다.

- 실행 대상 기기 또는 에뮬레이터를 선택한다.
- **app configuration**을 실행한다.

화면에서 보여줄 것:

- **MessageListScreen**
- 발신자
- 문자 미리보기
- AI 판단 배지
- 스미싱 확률
- 메시지 클릭 후 **MessageDetailScreen**
- 전체 본문
- ground truth label
- predicted label
- 정상 확률
- 스미싱 확률
- 모델 판단 안내 문구

말할 내용:

Android 앱은 문자 본문을 직접 분석하지 않습니다. **sample_messages.json**의 모델 확률을 **PrecomputedSampleClassifier**가 반환하고, UI는 그 classifier 결과만 표시합니다.

배지 기준:

- **phishingProbability < 0.40**: 정상
- **0.40 <= phishingProbability < 0.70**: 주의
- **phishingProbability >= 0.70**: 스팸 경고

설명:

이 threshold는 UI 표시용입니다. 텍스트에 어떤 단어가 들어 있는지 검사하지 않습니다.

6. Android classifier 구조 설명

예상 시간: 1분

보여줄 파일:

- **android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/SmishingClassifier.kt**
- **android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/PrecomputedSampleClassifier.kt**
- **android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/MockSmishingClassifier.kt**
- **android/IxissageApp/app/src/main/java/com/ixissage/app/classifier/ClassifierProvider.kt**
- **android/IxissageApp/app/src/main/java/com/ixissage/app/ui/MessageViewModel.kt**

말할 내용:

나중에 TFLite, LiteRT, ONNX Runtime Mobile 모델을 붙일 수 있도록 classifier 인터페이스를 분리했습니다. 지금은 `PrecomputedSampleClassifier`가 JSON에 저장된 확률을 반환하지만, 향후에는 같은 인터페이스를 구현하는 `LocalModelSmishingClassifier`로 교체할 수 있습니다.

구조:

```
sample_messages.json
-> MessageRepository
-> PrecomputedSampleClassifier
-> ClassificationResult
-> MessageViewModel
-> MessageListScreen / MessageDetailScreen
```

핵심:

- UI는 classifier 결과만 받는다.
- UI는 룰 기반 위험 판단을 하지 않는다.
- `MockSmishingClassifier`도 본문을 읽지 않는 고정 결과용 테스트 classifier다.

7. 온디바이스 추론 계획 설명

예상 시간: 1분

보여줄 파일:

- `ON_DEVICE_INFERENCE_PLAN.md`

말할 내용:

현재 학습 모델은 약 55MB이고 Hugging Face/PyTorch `safetensors` 형식입니다. Android에서 바로 실행하려면 ONNX, TFLite, LiteRT 같은 모바일 런타임 포맷으로 변환해야 합니다. 이때 모델 변환뿐 아니라 tokenizer 이식, operator 호환성, Python 결과와 Android 결과 비교 검증이 필요합니다.

추천:

- 발표용 1순위: `Precomputed inference demo`

대안:

- `ONNX Runtime Mobile`
- `LiteRT/TFLite` 변환

결론:

이번 발표에서는 안정적으로 AI 모델 흐름을 보여주기 위해 `precomputed inference` 방식을 사용하고, 실제 Android 온디바이스 추론은 후속 확장 과제로 남겼습니다.

5분 발표 버전

1. 프로젝트 목적과 룰 기반 탐지 금지 설명
2. 데이터셋과 supervised text classification 문제 설명

3. Transformer 평가 결과 설명
4. `src/inference.py`로 단일 문자 확률 출력 시연
5. Android 앱에서 샘플 문자 목록과 상세 화면 시연
6. classifier 인터페이스와 온디바이스 추론 후속 계획 설명

3분 발표 버전

1. "룰 기반이 아니라 AI 확률 기반 앱"이라고 소개
2. Transformer 모델 test F1 결과를 보여줌
3. Python 단일 추론 JSON 출력 시연
4. Android 목록/상세 화면 시연
5. 온디바이스 변환은 후속 과제라고 설명

발표 중 자주 받을 질문

실제 SMS를 읽나요?

아니다. 초기 버전은 실제 SMS 권한 없이 `sample_messages.json`을 사용한다. READ_SMS는 민감 권한이므로 후반부 선택 기능이다.

Android 앱에서 모델이 직접 돌고 있나요?

현재 발표 버전은 아니다. Python Transformer 모델의 inference 결과를 JSON에 저장하고 Android가 그 결과를 표시한다. Android 앱 구조는 향후 온디바이스 classifier로 교체할 수 있게 분리되어 있다.

URL이 있으면 스미싱이라고 판단하나요?

아니다. URL 포함 여부를 판단 feature로 쓰지 않는다. URL처럼 보이는 값은 샘플 안전을 위해 `https://example.com`으로 마스킹할 뿐이다. 위험 판단은 모델 확률만 사용한다.

키워드를 보고 판단하나요?

아니다. "택배", "미납", "환급", "본인인증" 같은 단어를 `if`문으로 검사하지 않는다. 모델은 문자 본문 전체를 tokenizer로 변환해 학습된 classifier로 판단한다.

실제 보안 제품으로 쓸 수 있나요?

아니다. 이 프로젝트는 수업 데모다. 데이터셋 라이선스와 실제 운영 환경의 위험을 고려하면 상용 보안 제품으로 사용하면 안 된다.

발표 체크리스트

- `outputs/transformer_report.md`가 존재한다.
- `models/smishing_classifier/model.safetensors`가 존재한다.
- `python3 src/inference.py --examples --include-ui-badge`가 실행된다.
- `android/IxissageApp/app/src/main/assets/sample_messages.json`에 30개 이상 샘플이 있다.
- `cd android/IxissageApp && ./gradlew build`가 성공한다.
- Android 앱 목록 화면에서 배지와 스미싱 확률이 보인다.
- 상세 화면에서 ground truth, predicted label, 정상 확률, 스미싱 확률이 보인다.

실패 시 대체 시연

기기나 에뮬레이터 실행이 안 될 때:

1. `./gradlew build` 성공 결과를 보여준다.
2. `sample_messages.json`의 확률 필드를 보여준다.
3. `MessageListScreen.kt`, `MessageDetailScreen.kt`가 classifier 결과를 표시하는 구조를 설명한다.
4. `src/inference.py --examples` 출력으로 모델 추론이 동작함을 보여준다.

모델 로딩이 느릴 때:

1. 이미 생성된 `outputs/inference_examples.md`를 보여준다.
2. 이미 생성된 `sample_messages.json`을 보여준다.
3. Android 앱에서는 precomputed inference 결과를 사용한다고 설명한다.

금지 사항

발표 중에도 다음 표현은 피한다.

- “URL이 있어서 스팸으로 판단했습니다.”
- “특정 키워드가 있어서 위험하다고 봅니다.”
- “이 앱은 실제 스미싱을 완벽히 막습니다.”
- “실제 사용자 SMS를 수집합니다.”

대신 다음처럼 설명한다.

- “모델이 계산한 `phishingProbability`가 높게 나왔습니다.”
- “배지는 모델 확률을 사람이 보기 쉽게 표시한 것입니다.”
- “이 앱은 수업 프로젝트용 AI 데모입니다.”
- “실제 SMS 접근 없이 샘플 데이터로 시연합니다.”